

B

Journey Orchestration Platform

Reusable journey-builder architecture (à la Alloy/Transmit/Strivacity): declarative journeys, server-driven UI, connector marketplace, and fraud-as-a-step — with a working engine prototype.

ARGUS FINANCIAL PARTNERS · CONFIDENTIAL — FOUNDER · JUNE 24, 2026

Strategic guidance only — not legal, financial, tax, or investment advice. Securities, money-transmission, and corporate-formation decisions must be reviewed by licensed counsel and a CPA before you act. Sections flagged “see counsel” are where this matters most.

Contents

1. 1. Honest review & gap analysis
2. 2. Pillar 1 — Declarative journey engine (prototyped)
3. 3. Pillar 2 — Server-Driven UI + multi-channel renderer (contract prototyped; renderers designed)
4. 4. Pillar 3 — Connector framework + vendor marketplace (prototyped)
5. 5. Pillar 4 — Risk/fraud as a journey node (prototyped)
6. 6. Cross-cutting — the "am I missing features?" answer
7. 7. What the prototype proves & the build roadmap

Journey Orchestration Platform — architecture review & design

How Argus becomes a reusable, configurable platform that can run **any account-opening flow** — a journey builder over a data layer, a vendor marketplace + bring-your-own-service, fraud/risk called at each step via a context, custom branding, and one journey rendered to any channel (web/iOS/Android) — benchmarked against **Alloy**, **Transmit Security**, **Strivacity**, with a working prototype of the core.

How to read this. A design doc (sibling to `PHASE-15-INTERNAL-AGENT-OPS.md`). The four pillars below are designed in full; the **declarative journey engine + connector framework + risk-node + SDUI contract** are **prototyped** (`backend/src/journeys/`, `test/journeys.test.ts`, 8 tests). Scope decision: **internal reusable engine** (not multi-tenant white-label yet) — but architected so productizing later is a seam, not a rewrite.

1. Honest review & gap analysis

Argus has excellent orchestration **primitives**, but was **not** architected as this platform. The benchmark vendors: **Alloy** (vendor orchestration + decisioning + waterfall/cascade + case mgmt), **Transmit** (CIAM + risk/DRS + orchestration), **Strivacity** (no-code journey builder + branding).

Capability	Argus before	Status	The platform
Flow definition	<code>riskOrchestratorService</code> — hardcoded TS	● → ● (proto)	Journey-as-data DAG (<code>journeys/types.ts</code>)
Orchestration runner	<code>WorkflowEngine</code> / <code>WorkflowDef</code> — imperative, headless , back-office	● substrate	Declarative runner over the DAG (<code>journeyRunner.ts</code>)
Conditions/branching	TS <code>if</code>	● → ●	CEL over the journey context (<code>journeys/cel.ts</code>)
Vendors	compile-time provider enums	● → ● (proto)	Runtime connector registry + BYO + waterfall (<code>connectors.ts</code>)
Fraud/risk	called on money events only	● → ● (proto)	<code>risk_check</code> step with journey context (<code>riskNode.ts</code>)
UI	hand-built per platform	● (design)	Server-Driven UI descriptors → thin renderers
Journey builder	none	● (design)	No-code editor over the DAG schema
Human-in-the-loop	<code>agent_reviews</code> queue	● reuse	<code>manual_review</code> step (pause/resume)
Versioning/rollout	fraud model-registry shadow/canary	● pattern	Journey versions + canary
Audit	append-only tables	● reuse	Append-only <code>journey_steps</code> trail
Resumability	Temporal/Conductor durable substrate	● substrate	Persisted run state (<code>journey_runs</code>)
Multi-tenant / branding	single-tenant; theming only	● (deferred)	Branding tokens now; tenancy later

Verdict: the substrate was strong; what was missing is the **declarative layer + the UI dimension + the runtime connector/risk model**. The prototype builds the engine core;

SDUI renderers and the visual builder are designed here and deferred.

2. Pillar 1 — Declarative journey engine (*prototyped*)

A **JourneyDef** is a versioned DAG: `{ id, version, start, steps: StepDef[] }`. Each **StepDef** is a typed node with `config` + routing (`next` and/or CEL `branches`). The **runner** (`journeyRunner.ts`) walks it over a **JourneyContext** (`data` + `connectorResults` + `riskDecisions` + `outcome`), recording an append-only step trail and persisting run state at every pause (fully **resumable** — start on web, finish on mobile).

- **Step-type registry** (`stepRegistry.ts`) — pluggable handlers: `collect · connector · risk_check · decision · branch · consent · manual_review · complete` (+ `sub_journey` for layered/composable flows, designed). New step types are a registry entry, not an engine change.
- **CEL conditions** (`journeys/cel.ts`) — the same non-Turing-complete subset as the fraud engine; a branch like `risk.kyc_risk.decision == 'deny'` routes the flow. Malformed CEL fails at **load** (`validateJourney`), never mid-run on a live applicant.
- **Versioning + canary** — reuse the fraud **model-registry shadow/canary** pattern: shadow a new journey version against the live one, compare, canary, promote — no redeploy.
- **Durable execution** — the runner can target the existing `WorkflowEngine` (Temporal/Conductor) for long-running, crash-safe journeys; the prototype persists to `journey_runs` directly.

Proven: the onboarding flow is now a **JourneyDef** (`onboardingJourney.ts`); editing a step's CEL threshold changes routing with **no code change** (`journeys.test.ts`).

3. Pillar 2 — Server-Driven UI + multi-channel renderer (*contract prototyped; renderers designed*)

The linchpin for "any UI." A `collect` / `consent` step emits a **channel-agnostic ScreenDescriptor** (`types.ts`): screen id, title, **typed fields** (text/email/date/select/checkbox/document/ **biometric**), validation (CEL), primary action, and **branding tokens** (accent/logo/theme). The journey defines the UI **once**; thin per-platform renderers interpret it:

```
JourneyDef → runner → ScreenDescriptor → [ web renderer | iOS renderer | Android  
└ branding tokens (white-label-ready, single-tenant)
```

Designed (not built): the three renderers, a component registry per platform, client-side validation from the CEL `validation`, i18n/accessibility. **Why SDUI:** it's the only way one journey definition serves web + iOS + Android without re-implementing each flow per platform — the property Transmit/ Strivacity sell. The prototype returns real `ScreenDescriptor`s so the contract is exercised end-to-end.

4 4. Pillar 3 — Connector framework + vendor marketplace (prototyped)

Argus's provider enums (`IDV_PROVIDER` , `SANCTIONS_PROVIDER` , ...) are **compile-time**. The platform needs **runtime connectors** (`connectors.ts`):

- **Registry** — connectors registered by id; a `connector` step names vendors by id (a builder UI could add one without a deploy).
- **Bring-your-own** — a **generic HTTP connector** (`httpConnector`) plugs any vendor/internal API or webhook in via `{ url, method, headers }`.
- **Waterfall / cascade** (`waterfall`) — try connectors in order, **first success wins**, every attempt recorded (Alloy's signature; auditable failover). *Proven:* the onboarding `verify_document` step cascades `always-fail → simulated` and records both attempts on the step trail.

Designed (not built): per-connector **secret vault** (reuse `keyVaultService`), request/response **field mapping**, cost/rate-limit + retry policy, a connector catalog UI. The existing provider seams wrap as connectors with a thin adapter.

5 5. Pillar 4 — Risk/fraud as a journey node (prototyped)

Exactly your "fraud called at each step via a context." A `risk_check` step (`riskNode.ts`) lifts signals from the **accumulated journey context** (via CEL maps), calls a `RiskProvider` , and the journey **branches on the decision + reason codes**:

```
risk_check(signals from ctx) -> { decision, score, reasonCodes } -> branches:
  risk.kyc_risk.decision == 'deny'    -> rejected
  risk.kyc_risk.decision == 'review' -> manual_review
  (default)                          -> consent (approve)
```

The default `RiskProvider` is simulated/offline; the **production swap** calls `fraudService` → `fraud-engine` (the same engine that scores money events, now a drop-in step) behind the same interface (`setRiskProvider`). Drop a `risk_check` anywhere — pre-collect device risk, post-document KYC, pre-funding fraud — passing whatever context has accumulated. *Proven:* sanctions → reject, elevated risk → review, clean → approve (`journeys.test.ts`).

6 6. Cross-cutting — the "am I missing features?" answer

Beyond your list, a complete platform needs these (each maps to an existing Argus primitive to reuse):

Missing capability	Why it matters	Reuse
Server-Driven UI (renderers)	the actual "any UI"	the <code>ScreenDescriptor</code> contract (built)
Connector secret vault + field mapping	real vendor integration	<code>keyVaultService</code> ; a mapping DSL (CEL)
Consent / e-sign + versioned disclosures	mandatory for account opening	<code>consent</code> step + a disclosures table
Save-&-resume / async + webhook-callback steps	long flows, vendor callbacks, review re-entry	<code>journey_runs</code> (built) + Temporal
Journey versioning + sandbox + simulation/replay	safe iteration	fraud model-registry shadow/canary
Analytics: per-step funnel / drop-off	conversion optimization	the append-only <code>journey_steps</code> trail
Composable sub-journeys ("layered")	reuse fragments across flows	a <code>sub_journey</code> step type (designed)
SDK / embeddable widget	drop into a partner/own app	the SDUI renderer + a JS SDK
PII governance (residency, retention, right-to-delete, field encryption)	compliance	<code>keyVaultService</code> + retention jobs
Idempotency + cost control + rate limiting on connector calls	money + spend safety	the idempotency middleware pattern
Outcome + reason-codes object	explainable decisions	built into <code>complete</code> + risk node
Per-step observability	ops	prom-client counters (add <code>journey_step_total</code>)
Decision/rules versioning + shadow	safe policy change	CEL + the model-registry pattern (proven)
Multi-tenancy + white-label + billing	productize to others	deferred — a tenant key on every table + per-tenant secrets/branding

7 7. What the prototype proves & the build roadmap

Built (`backend/src/journeys/` , 8 tests): journey-as-data DAG · step-type registry · CEL branching · risk-as-a-node · connector registry + BYO-HTTP + **waterfall** · SDUI `ScreenDescriptor` contract · resumable runs + append-only trail · the onboarding flow re-expressed as data. Decision-only and gated by `JOURNEYS_ENABLED` (it does not move money or grant tiers — shadow-style, like the fraud-CEL adoption).

Roadmap (internal-first):

1. **Now:** engine core (this prototype) → shadow the onboarding journey against `riskOrchestratorService` , compare outcomes, cut over.
2. **Next:** SDUI web renderer (then iOS/Android) over the `ScreenDescriptor` contract; connector **secret vault** + **field mapping**; consent/e-sign; per-step analytics.
3. **Then:** the **no-code journey-builder UI** over the `JourneyDef` schema; sub-journeys; sandbox + simulation/replay.
4. **Deferred (productize):** multi-tenancy, per-tenant branding/secrets/domains, SDK/widget, billing — the seams (branding tokens, runtime connectors, journey-as-data) are already in place so this is additive, not a rewrite.

Prototype:

`backend/src/journeys/{types,cel,stepRegistry,connectors,riskNode,journeyStore,journeyRunner,onboardingJourney}.ts` , `routes/journeys.ts` , migration `035_journeys.sql` , `test/journeys.test.ts` . Reuses the CEL subset (ported from `fraud-engine/src/rules/celEvaluator.ts`), the `agent_reviews` human-gate pattern, the fraud model-registry shadow/canary pattern, and append-only audit.